PYTHON

Classes and Objects

A complete, beginner-friendly guide with real-world examples, copy-paste code and clear outputs

CLASS Blueprint	**OBJECT** Real instance	**METHOD** Behaviour

Simple theory | Step-by-step explanation | Student, Employee and Bank examples

Prepared for Thaneshwara

1. Understanding Classes and Objects

Object-Oriented Programming (OOP) helps us represent real-world things in a program. A student, employee, bank account, product or patient can be represented using a class and objects.

The easiest way to remember

Class = Blueprint
Object = Real item created from the blueprint
Attribute = Object's data
Method = Object's behaviour

What is a Class?

A **class** is a blueprint or template used to create objects. It describes the common data and behaviour that every object of that type can have.

Real-world example

Student is a class. The class may define common details such as name, age and course. Rahul and Priya are individual students created from that blueprint.

What is an Object?

An **object** is a real instance created from a class. Every object can store its own values.

```
student1 = Student("Rahul", 21, "Python")
student2 = Student("Priya", 22, "Java")
```

Student is the class. **student1** and **student2** are objects. Both use the same blueprint, but they store different information.

CLASS	OBJECT
Blueprint or design	Real instance created from the blueprint
Example: Student	Example: Rahul or Priya
Defines common structure	Stores actual values
Created once	Many objects can be created

2. First Simple Program

This example creates a class, creates one object and calls a method.

```
class Student:

    def display(self):
        print("Welcome, Student!")


student1 = Student()
student1.display()
```

OUTPUT

```
Welcome, Student!
```

Line-by-line explanation

1	**class Student:**	Creates a class named Student.
2	**def display(self):**	Creates a method named display inside the class.
3	**print("Welcome, Student!")**	Prints a message when the method is called.
4	**student1 = Student()**	Creates an object named student1.
5	**student1.display()**	Uses the object to call the display method.

Important Python rule

The code inside a class must be indented. Python uses indentation to understand which methods and statements belong to the class.

3. Class with Variables

Variables that belong to an object are called **attributes**. In this program, each Student object stores a name and age.

```
class Student:

    def set_details(self):
        self.name = "Rahul"
        self.age = 21

    def display_details(self):
        print("Name:", self.name)
        print("Age:", self.age)


student1 = Student()
student1.set_details()
student1.display_details()
```

OUTPUT

```
Name: Rahul
Age: 21
```

What does self mean?

self = the current object

When **student1.set_details()** is called, **self** refers to student1. Therefore, **self.name** means the name stored inside student1.

Understanding the flow

Step	Code	What happens?
1	student1 = Student()	An empty Student object is created.
2	student1.set_details()	name and age are stored in student1.
3	student1.display_details()	The stored values are displayed.

4. Constructor: __init__()

A **constructor** is a special method that runs automatically when an object is created. In Python, the constructor is written as **__init__()**.

```
class Student:

    def __init__(self, name, age, course):
        self.name = name
        self.age = age
        self.course = course

    def display_details(self):
        print("Name:", self.name)
        print("Age:", self.age)
        print("Course:", self.course)


student1 = Student("Rahul", 21, "Python")
student1.display_details()
```

OUTPUT

```
Name: Rahul
Age: 21
Course: Python
```

Constructor explanation

Part	Meaning
__init__	Special constructor method; it runs automatically.
self	Refers to the current object being created.
name, age, course	Parameters receiving values during object creation.
self.name = name	Stores the received name inside the current object.

name versus self.name

name is a temporary parameter. **self.name** is an attribute stored permanently inside the object for later use.

5. Creating Multiple Objects

One class can create any number of objects. Each object has its own separate data.

```
class Employee:

    def __init__(self, employee_id, name, salary):
        self.employee_id = employee_id
        self.name = name
        self.salary = salary

    def display(self):
        print("Employee ID:", self.employee_id)
        print("Name:", self.name)
        print("Salary:", self.salary)
        print("--------------------")


employee1 = Employee(101, "Arun", 45000)
employee2 = Employee(102, "Sneha", 55000)

employee1.display()
employee2.display()
```

OUTPUT

```
Employee ID: 101
Name: Arun
Salary: 45000
--------------------
Employee ID: 102
Name: Sneha
Salary: 55000
--------------------
```

Key observation

employee1 and employee2 use the same Employee class, but their employee IDs, names and salaries remain separate.

6. Real-Time Example: Bank Account

This example combines attributes, a constructor and methods. It models simple bank operations.

```
class BankAccount:

    def __init__(self, account_number, customer_name, balance):
        self.account_number = account_number
        self.customer_name = customer_name
        self.balance = balance

    def deposit(self, amount):
        self.balance = self.balance + amount
        print(amount, "deposited successfully")

    def withdraw(self, amount):
        if amount <= self.balance:
            self.balance = self.balance - amount
            print(amount, "withdrawn successfully")
        else:
            print("Insufficient balance")

    def show_balance(self):
        print("Customer:", self.customer_name)
        print("Available balance:", self.balance)


account1 = BankAccount(1001, "Rahul", 10000)
account1.deposit(2000)
account1.withdraw(3000)
account1.show_balance()
```

OUTPUT

```
2000 deposited successfully
3000 withdrawn successfully
Customer: Rahul
Available balance: 9000
```

7. Bank Program - Step-by-Step Flow

Operation	Calculation	Balance
Opening balance	Starting value	10,000
Deposit	10,000 + 2,000	12,000
Withdraw	12,000 - 3,000	9,000

Understanding each method

deposit(amount)

Adds the deposited amount to the current balance.

withdraw(amount)

Checks whether enough balance is available. If yes, it subtracts the amount; otherwise, it prints an error message.

show_balance()

Displays the customer's name and the latest available balance.

Why classes are useful

Without a class, we may create many unrelated variables and functions. A class keeps related data and operations together. This makes programs easier to understand, reuse and maintain.

For example, a BankAccount class keeps account number, customer name, balance, deposit, withdrawal and balance display in one logical unit.

8. Important Terms - Quick Revision

Term	Meaning	Example
Class	Blueprint used to create objects	class Student:
Object	Instance created from a class	student1 = Student()
Attribute	Data stored inside an object	self.name
Method	Function defined inside a class	display()
Constructor	Runs automatically during object creation	__init__()
self	Refers to the current object	self.name

Final memory formula

Class = Blueprint
Object = Real item
Attribute = Data
Method = Behaviour
Constructor = Automatic setup
self = Current object

Practice task

Create a **Product** class with product_id, product_name and price. Add a method called **display_product()**. Create two product objects and display their details.

```
class Product:

    def __init__(self, product_id, product_name, price):
        # Write your code here
        pass

    def display_product(self):
        # Write your code here
        pass
```

You are ready!

You now understand the foundation of classes and objects in Python. The next useful topics are encapsulation, inheritance, polymorphism and abstraction.